Guia de Setup - StyleScan

Este documento fornece instruções detalhadas para configurar o ambiente de desenvolvimento do StyleScan.

📋 Pré-requisitos

Sistema Operacional

- Ubuntu 22.04 LTS (recomendado) ou macOS / Windows com WSL2

Software Necessário

- **Node.js** v22+ e npm
- **.NET SDK 8.0** ou superior
- **Git**
- **PostgreSQL 14+** ou **SQL Server 2019+**
- **Visual Studio Code** (recomendado) ou **Visual Studio 2022**

🔧 Instalação de Dependências

1. Node.js e npm

Ubuntu/Debian:

```bash
curl -fsSL https://deb.nodesource.com/setup_22.x | sudo -E bash -
sudo apt-get install -y nodejs
```

Verificar instalação:

```bash
node --version  # v22.x.x
npm --version   # 10.x.x
```

2. .NET SDK 8.0

Ubuntu/Debian:

```bash
sudo apt-get update
sudo apt-get install -y dotnet-sdk-8.0
```

macOS (com Homebrew):

```bash
brew install dotnet-sdk@8
```

Verificar instalação:

```bash
dotnet --version  # 8.0.x
```

3. Git

Ubuntu/Debian:

```bash
sudo apt-get install -y git
```

Configurar identidade global:

```bash
git config --global user.email "seu-email@example.com"
git config --global user.name "Seu Nome"
```

4. PostgreSQL (Recomendado)

Ubuntu/Debian:

```bash
sudo apt-get install -y postgresql postgresql-contrib

# Iniciar o serviço
sudo systemctl start postgresql
sudo systemctl enable postgresql
```

Criar usuário e banco de dados:

```bash
sudo -u postgres psql

# No prompt psql:
CREATE USER stylescan_user WITH PASSWORD 'password123';
CREATE DATABASE stylescan_dev OWNER stylescan_user;
ALTER ROLE stylescan_user SET client_encoding TO 'utf8';
ALTER ROLE stylescan_user SET default_transaction_isolation TO 'read committed'
ALTER ROLE stylescan_user SET default_transaction_deferrable TO on;
ALTER ROLE stylescan_user SET default_transaction_read_only TO off;
GRANT ALL PRIVILEGES ON DATABASE stylescan_dev TO stylescan_user;
\q
```

5. Ionic CLI

```bash
npm install -g @ionic/cli
```

Verificar instalação:

```bash
ionic --version
```

📦 Setup do Frontend

1. Navegar até o diretório frontend

```bash
cd /home/ubuntu/stylescan/frontend
```

2. Instalar dependências

```bash
npm install
```

3. Configurar variáveis de ambiente

Criar arquivo `src/environments/environment.ts`:

```typescript
export const environment = {
  production: false,
  apiUrl: 'http://localhost:5000/api',
  apiVersion: 'v1',
  openaiApiKey: process.env['OPENAI_API_KEY'] || '',
  replicateApiKey: process.env['REPLICATE_API_KEY'] || '',
  readyPlayerMeApiUrl: 'https://api.readyplayer.me',
};
```

Criar arquivo `src/environments/environment.prod.ts`:

```typescript
export const environment = {
  production: true,
  apiUrl: 'https://api.stylescan.app/api',
  apiVersion: 'v1',
  openaiApiKey: process.env['OPENAI_API_KEY'] || '',
  replicateApiKey: process.env['REPLICATE_API_KEY'] || '',
  readyPlayerMeApiUrl: 'https://api.readyplayer.me',
};
```

4. Executar servidor de desenvolvimento

```bash
ionic serve
```

O aplicativo estará disponível em `http://localhost:4200`.

5. Build para produção

```bash
ionic build --prod
```

Os arquivos compilados estarão em `www/`.

🖥️ Setup do Backend

1. Navegar até o diretório backend

```bash
cd /home/ubuntu/stylescan/backend
```

2. Restaurar dependências

```bash
dotnet restore
```

3. Configurar variáveis de ambiente

Criar arquivo `appsettings.Development.json` :

```json
{
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning"
    }
  },
  "AllowedHosts": "*",
  "ConnectionStrings": {
    "DefaultConnection": "Host=localhost;Port=5432;Database=stylescan_dev;Usern
  },
  "Jwt": {
    "SecretKey": "your-super-secret-key-that-is-at-least-32-characters-long-for
    "Issuer": "stylescan-api",
    "Audience": "stylescan-app",
    "ExpirationMinutes": 1440
  },
  "ExternalApis": {
    "OpenAI": {
      "ApiKey": "sk-your-openai-api-key",
      "Model": "gpt-4-vision-preview",
      "BaseUrl": "https://api.openai.com/v1"
    },
    "Replicate": {
      "ApiKey": "your-replicate-api-key",
      "BaseUrl": "https://api.replicate.com/v1"
    },
```

```
    "ReadyPlayerMe": {
      "ApiKey": "your-ready-player-me-api-key",
      "BaseUrl": "https://api.readyplayer.me"
    }
  },
  "Cors": {
    "AllowedOrigins": [
      "http://localhost:4200",
      "http://localhost:8100",
      "https://stylescan.app"
    ]
  }
}
```

4. Executar migrações do banco de dados

```bash
# Quando as migrações forem criadas
dotnet ef database update
```

5. Executar servidor de desenvolvimento

```bash
dotnet run
```

A API estará disponível em:

- HTTP: `http://localhost:5000`
- HTTPS: `https://localhost:7000`

6. Build para produção

```bash
dotnet publish -c Release -o ./publish
```

🔑 Configurar Chaves de API

OpenAI

1. Acesse https://platform.openai.com/api-keys
2. Crie uma nova chave de API
3. Adicione em `appsettings.Development.json` ou variável de ambiente

Replicate

1. Acesse https://replicate.com/account/api-tokens
2. Crie um novo token
3. Adicione em `appsettings.Development.json` ou variável de ambiente

Ready Player Me

1. Acesse https://dashboard.readyplayer.me
2. Crie uma aplicação
3. Obtenha a chave de API
4. Adicione em `appsettings.Development.json` ou variável de ambiente

🧪 Testar a Instalação

Frontend

Bash

```bash
cd frontend

# Testar se o servidor inicia
ionic serve

# Em outro terminal, testar se está respondendo
curl http://localhost:4200
```

Backend

Bash

```bash
cd backend

# Testar se o servidor inicia
dotnet run
```

```
# Em outro terminal, testar o endpoint de health
curl https://localhost:7000/health -k
```

🚀 Próximos Passos

1. **Criar modelos de dados:** Definir entidades no backend
2. **Configurar banco de dados:** Criar migrações iniciais
3. **Implementar autenticação:** JWT no backend e guards no frontend
4. **Desenvolver componentes base:** Layouts e componentes reutilizáveis
5. **Integrar APIs de IA:** Conectar OpenAI e Replicate

🆘 Troubleshooting

Erro: "npm: command not found"

Bash

```
# Reinstalar Node.js
curl -fsSL https://deb.nodesource.com/setup_22.x | sudo -E bash -
sudo apt-get install -y nodejs
```

Erro: "dotnet: command not found"

Bash

```
# Reinstalar .NET SDK
sudo apt-get install -y dotnet-sdk-8.0
```

Erro: "ionic: command not found"

Bash

```
# Reinstalar Ionic CLI
npm install -g @ionic/cli
```

Erro de conexão com PostgreSQL

Bash

```
# Verificar se o serviço está rodando
sudo systemctl status postgresql

# Iniciar se necessário
sudo systemctl start postgresql

# Verificar credenciais em appsettings.Development.json
```

Porta já em uso

Se a porta 5000 (backend) ou 4200 (frontend) já está em uso:

Frontend:

```bash
ionic serve --port 4300
```

Backend:

```bash
dotnet run --urls "http://localhost:5001;https://localhost:7001"
```

📚 Recursos Adicionais

- Ionic Documentation
- Angular Documentation
- .NET 8 Documentation
- Entity Framework Core
- PostgreSQL Documentation

Última atualização: 21 de março de 2026